

# REQMEMBENCH: BENCHMARKING CODING AGENTS ON TEMPORAL REQUIREMENT-STATE RECONSTRUCTION

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Coding agents increasingly operate as persistent participants in software projects, inheriting evolving codebases midway through long-running client collaborations. Successful continuation requires recovering the specification governing the current task: requirements may have been revised, narrowed in scope, deferred, resumed, or withdrawn across earlier interactions. Existing coding and memory benchmarks offer limited visibility into whether agents correctly reconstruct the requirement state that should guide implementation. We introduce ReqMemBench, a benchmark for temporal requirement-state reconstruction built from real freelance project histories that have been rewritten for privacy and include a subset of role-played messages. At intermediate takeover points, ReqMemBench provides evidence-linked gold requirement states and evaluates four capabilities: selecting relevant historical requirements and evidence, reconstructing their pre-task states, updating requirements or identifying material clarification needs, and delivering the requested code changes. A two-phase protocol freezes history-based responses before agents choosing to act on eligible tasks receive repository access. Full History and Oracle Relevant History conditions assess the effect of history filtering and agents’ reconstruction accuracy when relevant evidence is explicitly supplied. [TO COMPLETE: report the principal quantitative findings, including performance under oracle history and the relationship between requirement-state accuracy and code delivery.]

## 1 INTRODUCTION

Code-generation evaluation has progressively expanded from self-contained function synthesis to repository-scale software engineering. HumanEval and MBPP measure whether models can produce short programs from natural-language specifications (Chen et al., 2021; Austin et al., 2021). CrossCodeEval and RepoBench add cross-file retrieval and repository context (Ding et al., 2023; Liu et al., 2024), while SWE-bench evaluates patches for real GitHub issues and SWE-Lancer evaluates freelance engineering tasks with executable or expert-derived criteria (Jimenez et al., 2024; Misrendino et al., 2025). These benchmarks broaden the scope and realism of coding evaluation, but do not explicitly measure whether agents can reconstruct requirement states from histories of revisions, deferrals, and withdrawals. A coding agent joining an ongoing project faces a prior question: before deciding how to modify the code, it must determine which requirements currently govern the task.

The information needed to reconstruct these requirements is distributed across the project’s interaction history. Requirements engineering has long emphasized tracing requirements from their origins through refinement and implementation (Nuseibeh & Easterbrook, 2000; Gotel & Finkelstein, 1994). In a long-running project, a requirement may be introduced, modified, deferred, resumed, or removed. Some aspects may remain ambiguous while others are settled, and execution feedback may update the known implementation status without changing the requirement itself. A previously authoritative statement may therefore be superseded in whole or in part at takeover, yet remain useful evidence for reconstructing how a requirement evolved. The central challenge is to reconstruct the current state of task-relevant requirements from these updates, preserving both settled constraints and unresolved ambiguities.

Several related lines of evaluation test important parts of this challenge. Conversational coding benchmarks measure how models revise programs in response to iterative compilation, execution, or verbal feedback (Han et al., 2025). Long-term memory benchmarks test capabilities including information extraction, temporal reasoning, knowledge updates, long-range understanding, selective forgetting, and abstention over extended interactions (Wu et al., 2025; Maharana et al., 2024; Hu et al., 2025). These benchmarks assess the use of historical information, and some also evaluate retrieval quality. However, they do not directly score the task-relevant project requirement state that should guide code implementation, including requirement values, lifecycle status, scope, unresolved ambiguities, and implementation evidence. Explicit evaluation of this state would support separate assessment of history selection, requirement reconstruction, and code delivery.

We formulate this setting as *temporal requirement-state reconstruction*. An agent enters a project at a target time  $t^*$  and first reasons from the condition-specific pre-target history  $H_{<t^*}^{(c)}$  and current task  $q_{t^*}$ . During this history-only phase, it selects the directly affected historical requirements and their supporting evidence, reconstructs their pre-task states, and either predicts complete post-task states for all affected requirements (ACT) or identifies material ambiguities requiring clarification (CLARIFY). The response is frozen before repository access is granted. For an execution-eligible target, a run proceeds to an execution phase with the pre-task repository  $C_{t^*}$ —only when the agent’s frozen decision is ACT. This separation protects the history-only evaluation from repository-derived information and allows the frozen requirement interpretation to be compared with final code correctness.

ReqMemBench operationalizes this formulation by annotating longitudinal project histories as evidence-linked Requirement Events and replaying validated events into temporal Requirement State Graphs. Each takeover instance includes task-local gold pre-task states and either complete post-task states or clarification targets, execution-eligible instances additionally have a fixed hidden validator. The benchmark evaluates four capabilities along the path from history to delivery: selecting directly affected historical requirements and supporting evidence, reconstructing their pre-task states, producing complete post-task states or identifying material clarification needs, and delivering the requested code changes. Full History provides the complete transformed pre-target conversation, whereas Oracle Relevant History provides an audited, order-preserving subsequence retaining the relevant requirement trajectories and context, including superseded statements where needed. Requirement and evidence selection are scored only under Full History because oracle construction uses gold relevance annotations. The oracle condition evaluates reconstruction and update-or-clarify reasoning with relevant evidence explicitly supplied.

Our contributions are:

- We formulate temporal requirement-state reconstruction as an evaluation problem for coding-agent takeover, with explicit task-local targets for reconstructing pre-task requirement states and determining their updates or material clarification needs under the current request.
- We construct ReqMemBench from transformed longitudinal freelance project histories, providing evidence-linked Requirement Events, temporal Requirement State Graphs, and annotated takeover instances at intermediate project stages.
- We design a two-phase protocol that freezes history-based responses before repository access and compares requirement reconstruction, update-or-clarify reasoning, and eligible code delivery under Full History and Oracle Relevant History.

## 2 RELATED WORK

### 2.1 CODING AND SOFTWARE ENGINEERING BENCHMARKS

Coding benchmarks span function-level synthesis in HumanEval and MBPP (Chen et al., 2021; Austin et al., 2021) and library-intensive programming in BigCodeBench (Zhuo et al., 2025). Cross-CodeEval and RepoBench evaluate code completion with cross-file context (Ding et al., 2023; Liu et al., 2024), while SWE-bench and SWE-Lancer cover issue resolution and freelance engineering tasks (Jimenez et al., 2024; Miserendino et al., 2025). SWE-EVO evaluates release-level software evolution (Le et al., 2025), and ProjDevBench evaluates project creation and completion (Lu et al.,

2026). Beyond implementation outcomes, RigorBench evaluates engineering process discipline, including abstention and clarification on infeasible or ambiguous tasks (Madiraju & Madiraju, 2026).

## 2.2 LONG-TERM MEMORY AND LONG-CONTEXT EVALUATION

An emerging line of research evaluates agents in more realistic and complex settings involving sequences of tasks, where prior work can serve as a resource. SWE-Bench-CL (Joshi et al., 2025), SWE Context Bench (Zhu et al., 2026), and SlopCodeBench (Orlanski et al., 2026) evaluate knowledge transfer, trajectory reuse, and quality degradation, respectively. In these settings, history is treated as potentially useful experience. The other group of long-term memory benchmarks treat history as a store of facts that an assistant must retain, update, and retrieve on demand. LongMemEval (Wu et al., 2025), MemoryAgentBench (Hu et al., 2025), and LoCoMo (Maharana et al., 2024) test retention over long multi-session dialogue, including knowledge updates, selective forgetting, and abstention-analogues of requirement override and clarification. These benchmarks evaluate whether an agent can use the history and the current query to recover an answer already contained in that history. However they fail to question whether an agent can integrate project history, existing code, and the current task to reconstruct the project state and determine the appropriate action. This state is rarely stated explicitly: it must be inferred from requirements with different scopes and validity, and must then guide the implementation.

## 2.3 LONG-HORIZON CONVERSATIONAL CODING

Multi-turn coding benchmarks allow the target to be clarified or modified throughout an interaction, making them the closest to real-world development settings. ConvCodeWorld (Han et al., 2025) and RECODE-H (Miao et al., 2026) evaluate code revision under feedback toward a fixed target. SR-Eval (Zhan et al., 2025) simulates the stepwise refinement of a final requirement, while LoCoEval (Liu et al., 2026) examines context management in LLM-generated repository conversations involving iterative requirements.

Taken together, the benchmarks discussed above and long-horizon conversational coding benchmarks capture complementary aspects of the broader problem. However, none explicitly treats the evolving validity and scope of requirements over time-the temporal requirement state defined in ReqMemBench-as the object of evaluation. Our comparison therefore focuses on differences in the inputs provided to agents, the gold targets used for evaluation, and the outputs agents are expected to produce, while fully acknowledging the distinct contributions of prior work.

## 3 TEMPORAL REQUIREMENT-STATE RECONSTRUCTION

Requirements engineering has long treated requirements as artifacts whose origins, refinements, and realization must remain traceable across a system’s lifecycle (Nuseibeh & Easterbrook, 2000; Gotel & Finkelstein, 1994). ReqMemBench uses evidence-linked requirement states as explicit evaluation targets at intermediate project takeover points. Given the preceding interaction history and current request, the agent reconstructs task-relevant pre-task states and either predicts complete post-task states for all affected requirements or identifies material ambiguities requiring clarification.

### Figure 1 placeholder: Motivation and task definition

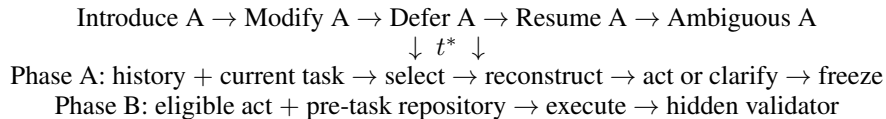


Figure 1: The ReqMemBench takeover setting. An agent first reasons from the preceding history and current task without repository access. After its response is frozen, eligible ACT runs receive the pre-task repository for delivery evaluation.

Table 1: Positioning ReqMemBench relative to related evaluation directions. All entries must be verified against the cited work.

| Dimension                  | Coding | Memory | Conv. coding | ReqMemBench |
|----------------------------|--------|--------|--------------|-------------|
| Real repository            | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Longitudinal project       | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Historical conversation    | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Requirement evolution      | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Explicit lifecycle         | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Scope and ambiguity        | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Temporal gold state $G(t)$ | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Intermediate takeover      | [TBD]  | [TBD]  | [TBD]        | Yes         |
| Downstream action          | [TBD]  | [TBD]  | [TBD]        | Yes         |

### 3.1 PROBLEM FORMULATION

We represent a project as  $P = (H, C)$ , where  $H = \langle h_1, \dots, h_n \rangle$  is the temporally ordered interaction history and  $C(t)$  is the corresponding code state. The history contains client and developer messages together with recorded execution feedback. A target task  $q_{t^*}$  arrives at takeover time  $t^*$ , inducing a strict temporal boundary:  $H_{<t^*}$  contains only interactions before the target, and  $C_{t^*-}$  denotes the code snapshot immediately before it. No evidence produced after the target may enter the pre-task state, the target itself is used only to identify which historical requirements are relevant.

The atomic unit of state is a *Requirement Atom*. Atoms that concern the same feature may be grouped into a Requirement Family, but each atom has an independent lifecycle. For an atom  $r$ , we define its state at time  $t$  as

$$G_r(t) = (A_r(t), L_r(t), S_r(t), U_r(t), X_r(t)), \quad (1)$$

where  $A_r$  is the map of currently valid attributes.  $L_r$  is the lifecycle state, such as ACTIVE, DEFERRED, or REMOVED.  $S_r$  records persistence, component, and contextual scope.  $U_r$  is the set of unresolved ambiguities and their affected dimensions. And  $X_r$  records the latest implementation evidence, such as CLAIMED\_WORKING, FAILED, or VERIFIED\_WORKING. Unknown fields remain unknown rather than being completed by inference.

The five dimensions are deliberately separate. An active requirement may contain an unresolved ambiguity, and a valid requirement may currently have a failed implementation. Conversely, removing a requirement changes its lifecycle but does not erase its historical attributes or execution evidence. CLARIFY is therefore an agent decision introduced below, not a lifecycle value or a requirement event.

ReqMemBench derives states by replaying temporally ordered Requirement Events. Let

$$E_r = \langle e_r^{(1)}, \dots, e_r^{(n_r)} \rangle \quad (2)$$

be the validated event sequence for atom  $r$ . Each event deterministically transforms the preceding state,

$$S_r^{(k)} = T(S_r^{(k-1)}, e_r^{(k)}), \quad G_r(t) = \text{Replay}(E_r^{\leq t}), \quad (3)$$

where  $T$  implements the transition rules described in Section 4. The event vocabulary separates semantic changes (INTRODUCE, MODIFY, DEFER, RESUME, and REMOVE), uncertainty (AMBIGUOUS), and implementation evidence (IMPLEMENTATION\_CLAIM, RUNTIME\_FAILURE, and RUNTIME\_VERIFICATION). Every state retains the events and source messages that currently support its fields, so replay produces both a snapshot and its temporal provenance.

The project state is the collection of all requirement states observable by time  $t$ ,

$$G_P(t) = \{G_r(t) \mid r \in R_P(t)\}. \quad (4)$$

Removed requirements remain in  $G_P(t)$  with lifecycle REMOVED, otherwise an evaluator could not distinguish correct invalidation from accidental resurrection. ReqMemBench does not ask an agent to reproduce this entire project graph for every target. Let  $A_{t^*}$  be the atoms directly affected by

the target, including atoms first introduced by it, and let  $R_P(t^{*-})$  be the atoms that existed before takeover. The task-relevant historical set and its pre-task state are

$$R_{t^*}^{\text{hist}} = A_{t^*} \cap R_P(t^{*-}), \quad G_{t^*}^- = \{G_r(t^{*-}) \mid r \in R_{t^*}^{\text{hist}}\}. \quad (5)$$

Atoms introduced by  $q_{t^*}$  are absent from  $G_{t^*}^-$ . They enter the complete post-task target together with affected historical atoms:

$$G_{t^*}^+ = \{G_r(t^{*+}) \mid r \in A_{t^*}\}. \quad (6)$$

This projection makes selection, reconstruction, and updating measurable without requiring recovery of unrelated parts of the project.

### 3.2 TAKEOVER TASK AND EVALUATION STAGES

Evaluation uses two history conditions. C1 exposes the full pre-target history,  $H_{<t^*}^{(C1)} = H_{<t^*}$ . C2 exposes an audited, order-preserving subsequence  $H_{<t^*}^{(C2)} \subseteq H_{<t^*}$  that retains the trajectories and contextual messages needed to determine the same gold state. Both conditions represent takeover of an ongoing project, neither removes history altogether.

Each logical run has two phases. In Phase A, the repository is absent and the agent receives only condition-specific history and the current task:

$$H_{<t^*}^{(c)} + q_{t^*} \longrightarrow (\hat{R}_{t^*}^{\text{hist}}, \hat{G}_{t^*}^-, \hat{d}_{t^*}, \hat{Z}_{t^*}), \quad (7)$$

where  $c \in \{C1, C2\}$ ,  $\hat{d}_{t^*} \in \{\text{ACT}, \text{CLARIFY}\}$ , and  $\hat{Z}_{t^*}$  is either a complete predicted post-task state  $\hat{G}_{t^*}^+$  or a structured set of blocking issues and clarification questions  $\hat{Q}_{t^*}$ . An open ambiguity warrants CLARIFY only when it is material to the target, unresolved by the visible evidence, and prevents a unique safe update.

The Phase-A response is frozen before code becomes visible. For an RQ4-eligible target whose gold decision is ACT, the agent enters Phase B only if its frozen decision is also ACT:

$$H_{<t^*}^{(c)} + q_{t^*} + C_{t^*}^- + \text{FrozenResponse}_{t^*}^{(c)} \longrightarrow \hat{C}_{t^*+}. \quad (8)$$

This boundary prevents the repository, build output, or hidden tests from revealing the answers to the history-only tasks. It also separates an incorrect interpretation from an implementation failure under a correct interpretation.

RQ1 is evaluated only under C1 because C2 is constructed from the relevant gold trajectory and would expose the selection answer. RQ2 scores state correctness only over successfully aligned historical atoms and reports reconstruction coverage separately, preventing selection errors from being counted twice. RQ3 evaluates all affected atoms, including those introduced by the target, and requires either a complete update or complete coverage of the material blockers. RQ4 evaluates only the final repository: a delivery passes when build, target-specific tests, and regression tests all pass. Together, the stages localize failure along the chain

$$\text{Select} \rightarrow \text{Reconstruct} \rightarrow \text{Decide} \rightarrow \text{Execute}. \quad (9)$$

Table 2: The four evaluation stages. C1 is Full History and C2 is Oracle Relevant History.

| Stage | Evaluated capability                                                       | Gold target                                            | History        | Repository        |
|-------|----------------------------------------------------------------------------|--------------------------------------------------------|----------------|-------------------|
| RQ1   | Select directly affected historical atoms and their current-state evidence | $R_{t^*}^{\text{hist}}$ and supporting evidence groups | C1             | No                |
| RQ2   | Reconstruct the pre-task state of aligned historical atoms                 | $G_{t^*}^-$ over matched atoms                         | C1/C2          | No                |
| RQ3   | Construct the complete post-task state or identify material blockers       | $G_{t^*}^+$ for ACT, or clarification gold             | C1/C2          | No                |
| RQ4   | Deliver the frozen update in code                                          | Final repository passing the frozen validator          | Eligible C1/C2 | Yes, after freeze |

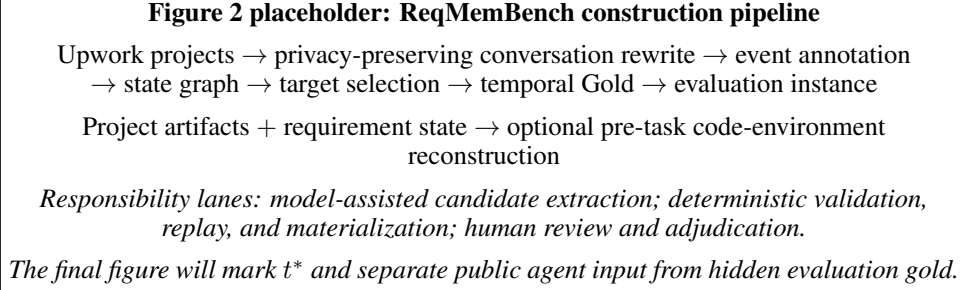


Figure 2: The ReqMemBench construction pipeline from project source material to temporal requirement-state evaluation instances.

## 4 REQMEMBENCH CONSTRUCTION

ReqMemBench construction separates the provenance of project histories, requirement annotations, and executable code environments. It transforms the source material into temporal requirement graphs, selects intermediate takeover points, and materializes condition-controlled evaluation instances while keeping researcher-side Gold and execution artifacts hidden from the agent.

### 4.1 DATA AND TEMPORAL REQUIREMENT GRAPH CONSTRUCTION

**Project provenance and privacy transformation.** ReqMemBench is derived from software projects conducted through Upwork, including project descriptions, deliverables and other available artifacts, and conversations between clients and freelancers. The source projects and their interactions are real, but the conversations released to the construction pipeline are not verbatim platform transcripts. We rewrite every conversation to protect the participants’ privacy. The transformation replaces or removes direct identifiers, private links, account information, credentials, and project-specific entities while preserving the requirements, decisions, negations, temporal order, cross-message revisions, and reported execution outcomes needed for temporal reasoning. Replacements are planned consistently at the project level, checked against the source meaning, and subjected to a final safety audit. A conversation is withheld if any message remains unresolved. A small subset of messages is role-played. ReqMemBench should therefore be understood as a privacy-preserving transformation of real project histories that contains a limited role-played component, rather than as a release of raw conversations.

Code provenance is tracked separately from conversation provenance. Available project artifacts provide evidence about the implemented system and its toolchain. When an exact executable snapshot immediately before a target is unavailable, we reconstruct a pre-task code environment from the available artifacts and the annotated requirement state. Each reconstructed environment is aligned to a specific target boundary, built deterministically, and validated for reproducibility and consistency with the corresponding pre-task state. Recovered artifacts and reconstructed executable environments are distinguished in researcher-side metadata. A reconstructed environment is not treated as an observed historical repository snapshot. *[TODO: report the frozen counts of projects and messages, the rewritten-versus-role-played provenance breakdown, and the recovered-versus-reconstructed code-environment breakdown after the release snapshot is fixed.]*

**Requirement-event annotation.** The transformed history is converted into a temporal representation in two stages. Stage 1 first groups semantically related requirements into optional Requirement Families and then segments them into Requirement Atoms, each of which has an independent lifecycle. It next associates state-changing evidence with exactly nine first-class Requirement Events:

$$\mathcal{E} = \{\text{INTRODUCE, MODIFY, DEFER, RESUME, REMOVE, AMBIGUOUS, IMPLEMENTATION\_CLAIM, RUNTIME\_FAILURE, RUNTIME\_VERIFICATION}\}.$$

Each event is anchored to a source message and the normalized conversation order, identifies its owning atom, and records the payload needed to update attributes, scope, lifecycle, ambiguity, or

execution state. CLARIFY is an agent decision in RQ3 rather than a Requirement Event. Likewise, ambiguity resolution is represented by an eligible later state-changing event that references the ambiguity it resolves, rather than by a separate RESOLVE event. Model-assisted extraction proposes families, atoms, events, and evidence links; schema checks, cross-reference validation, and review determine which candidates enter the canonical annotation.

**Deterministic replay and state graphs.** Stage 2 treats the validated event sequence as canonical input and does not reinterpret the conversation. For each atom, it orders events by their normalized message position and deterministically applies the transition function in Equation 3. Every resulting node contains the complete five-dimensional state from Equation 1, while the incoming edge records the event responsible for the transition. Supporting-event references are updated with the state, preserving a trace from each current field to its historical evidence. Schema violations, invalid transitions, and unresolved cross-references cause validation failure rather than model-based repair. The atom-level trajectories are then combined into one project graph that can be replayed at multiple takeover times, removed requirements remain represented with lifecycle REMOVED.

## 4.2 TAKEOVER INSTANCE CONSTRUCTION AND HISTORY CONDITIONS

**Selecting takeover points.** A target candidate is a client message, not an individual event. All events triggered by the same message are grouped into one candidate so that a multi-requirement request remains one takeover task. The candidate pool prioritizes state-changing messages involving modification, removal, deferral, resumption, ambiguity, or ambiguity resolution. An INTRODUCE event may also qualify when it occurs after an existing project state and therefore depends on prior context, whereas the first context-free introduction and messages containing only execution evidence are excluded by default. Candidate packets contain the current message, its triggered events, the affected atoms’ pre-task states and earlier trajectories, and their historical evidence. They exclude future messages and future-supported states.

A structured model assessment rates each candidate’s historical dependency, requirement evolution, reconstruction risk, ambiguity-decision value, and multi-requirement value. These categorical ratings are converted deterministically into a selection score. Retained candidates must be valid development tasks, depend on history, and be recommended under the frozen selection rule. Stable IDs, message order, event grouping, score calculation, and all Gold states are produced by code rather than by the selector. Each selected target records whether it entered through threshold-based selection or human review so that the final benchmark can report the provenance of target selection. *[TODO: report the frozen selection threshold and review route, together with candidate, accepted, rejected, and add-back counts.]*

**Gold boundaries and task-local projections.** For a selected target  $q_{t^*}$ , the pre-task Gold state is obtained by replaying only events whose source messages occur strictly before the target. The post-task Gold state additionally applies every event triggered by the target message, in their validated order. Thus the two boundaries are exclusive and inclusive with respect to the same message:

$$G_P(t^{*-}) = \text{Replay}(E^{<t^*}), \quad G_P(t^{*+}) = \text{Replay}(E^{\leq t^*}).$$

An atom first introduced by the target has no pre-task state, an atom removed by the target remains in the post-task graph with lifecycle REMOVED. Construction stores complete project snapshots at both boundaries and then derives the target-local sets in Equations 5 and 6. This order avoids constructing Gold from the agent-facing history condition.

**RQ-specific construction.** The instance constructor joins four researcher-side sources: the target and its pre/post state references, the project state graph, the transformed pre-target conversation, and, for RQ4 candidates only, a target-aligned code environment. Applicability is derived deterministically rather than inherited from model-generated RQ tags. A target enters RQ1 and RQ2 when it directly affects at least one atom that existed before takeover. It enters RQ3 when the target induces at least one verifiable requirement transition, and it becomes an RQ4 candidate only when such a transition has a matching pre-task code environment.

RQ1 Gold contains the directly affected historical atoms and deterministic evidence groups supporting their current pre-task fields. RQ2 expands the pre-task states only for those historical atoms,

requirements first introduced by the target are recorded separately and never receive a fabricated pre-state. RQ3 represents either the complete post-task states of all affected atoms or the material blocking ambiguities and corresponding clarification questions. C1 and C2 share the same frozen RQ3 branch and semantic Gold. An RQ4 candidate becomes formally eligible only when its RQ3 Gold decision is ACT, its pre-task environment is reproducibly executable, the target has locally observable acceptance criteria, the hidden validator has been reviewed and calibrated, and repository-leakage checks pass.

**History conditions and leakage control.** Every instance contains a common pool of transformed messages strictly before the target. C1 selects the entire pool as Full History. C2 selects an order-preserving Oracle Relevant History consisting of the affected atoms’ trajectories and the contextual messages needed to interpret them. RQ1 is scored only under C1 because constructing C2 uses the RQ1 Gold trajectory, exposing C2 to RQ1 would reveal the selection answer. RQ2 and RQ3 use both conditions, and eligible RQ4 runs inherit the same condition-specific history. No-History is not part of the benchmark.

The materializer merges the RQ-specific researcher views into one logical run for each target-condition pair. Phase A exposes only the target task, selected history, common instructions, and a response schema through an opaque workspace. It withholds condition names, RQ eligibility, internal Requirement/Event/State identifiers, construction Gold, code paths, repositories, validators, and future information. After the structured RQ1–RQ3 response is frozen and hashed, an eligible run whose agent decision is ACT enters Phase B. The runner then extracts a fresh copy of the target-aligned pre-task repository, including reconstructed environments where applicable, and supplies a read-only copy of the frozen response. C1 and C2 use the same repository hash, modified repositories are never reused across runs, and the hidden validator executes outside the agent workspace. These controls prevent code or test feedback from changing the history-only answers. *[TODO: insert the frozen RQ1–RQ4 instance counts by condition and the final numbers of RQ4 candidates, executable environments, eligible runs, and exclusions.]*

#### 4.3 ANNOTATION QUALITY AND BENCHMARK STATISTICS

**Annotation and quality control.** The final description will separate the following responsibilities:

- **Language-model assistance:** propose requirement families, atoms, events, source evidence, or candidate target times, with all outputs treated as candidates subject to validation and review.
- **Deterministic processing:** sort events, validate schemas, replay events, generate states, join histories, code, and targets, test consistency, detect future leakage, and reject impossible transitions.
- **Human review:** apply stage-specific review. RQ1 uses deterministic gold and calibrated semantic alignment. RQ2 requires typed-field and comparator review. RQ3 requires two independent reviewers plus adjudication. RQ4 requires acceptance-criterion and hidden-validator review and calibration.
- **Evidence retention:** trace each event and transition to source evidence and a temporal-order anchor.
- **Quality reporting:** report the review sample, annotator count, training and frozen guidelines, adjudication, field-level agreement, and suitable agreement statistics.

A generic agreement sample will not be presented as validation for all four RQs. Any retained audit of 200 state-change steps will be reported only for the construction fields it examines, separately from RQ1 judge calibration, RQ3 gold adjudication, and RQ4 validator calibration. *[TODO: insert the actual annotation workflow, review scale, agreement results, error-correction procedure, and scorer/judge calibration.]*

**Curation-validity checks.** Requirement determinacy will be reported separately for each construction layer. Stage-1 and state-graph audits test event and transition consistency, RQ2 review freezes typed fields and comparators, RQ3 uses independent review and adjudication, and RQ4 validates acceptance criteria and hidden tests against the pre-task repository, reference solution, and

Table 3: Planned ReqMemBench statistics. Reported construction counts remain provisional until regenerated from a frozen dataset snapshot.

| Level        | Required statistics                                                                  | Current snapshot       |
|--------------|--------------------------------------------------------------------------------------|------------------------|
| Project      | Count, provenance, duration, messages, history length, and repository recoverability | 51 projects            |
| Requirement  | Families, atoms, requirements per project, and populated dimensions                  | 859 atoms              |
| Event        | Count, type distribution, events per requirement, and transitions                    | 2,793 events           |
| Task         | Count, source projects, affected requirements, target position, and history length   | 210 tasks; 39 projects |
| RQ/condition | Constructed, reviewed, eligible, and evaluated instances for RQ1–RQ4 under C1/C2     | [TBD]                  |
| Difficulty   | Short, medium, and long counts by project and RQ                                     | [TBD]                  |

**Figure 3 placeholder: benchmark distributions**

Project duration and history length; requirements per project; events per requirement; target positions; affected requirements; and RQ/condition/difficulty coverage.

Figure 3: Planned ReqMemBench scale and distribution summary. The final figure will expose empty or heavily imbalanced strata.

applicable partial deliveries. Oracle-history sufficiency will verify that C2 is an ordered subsequence of C1 and preserves every message needed to determine the same RQ2 pre-task state and RQ3 gold branch. A C1/C2 gold disagreement indicates missing oracle evidence or a gold-review error, not a condition effect.

**Benchmark statistics.** Table 3 will report totals with appropriate means, medians, ranges, or quantiles.

## 5 EXPERIMENTS

We evaluate whether a coding agent can carry an evolving specification from historical evidence to a validated repository. The evaluation unit is a model–harness pair, and the logical run unit is a target–condition pair. One frozen Phase-A response supplies all applicable RQ1–RQ3 outputs. Phase B begins only when the RQ3 decision is ACT and the target passes the RQ4 eligibility gate. This design preserves the dependency between the four capabilities without rerunning the same agent separately for each research question.

### 5.1 EXPERIMENTAL SETUP

**Conditions and protocol.** C1 provides the complete transformed pre-target conversation, whereas C2 provides an order-preserving oracle subset containing the affected requirements’ trajectories and the context needed to interpret them. RQ1 is evaluated only under C1 because C2 is constructed from the gold-relevant trajectory and would reveal the selection answer. RQ2 and RQ3 use both conditions. RQ4 inherits the corresponding condition-specific history but exposes the same pre-task repository only after Phase A has been frozen. No-History is not a formal condition.

For a given target, C1 and C2 use the same current task, response schema, model, prompt, non-repository tools, and resource budget. Each condition and replicate runs in a fresh workspace without cross-run session memory. Condition labels, research-question labels, gold states, internal requirement identifiers, and evaluator artifacts are hidden from the agent. In Phase B, both conditions additionally share the same repository snapshot, execution tools, budget, and target-specific validator. History length is analyzed, rather than sampled, using Short (0–25 prior turns), Medium (26–50), and Long (> 50) strata.

Table 4: Primary metrics and comparison sets for the four evaluation stages.

| Stage | Main-text metrics                                            | Scoring set or comparison                       |
|-------|--------------------------------------------------------------|-------------------------------------------------|
| RQ1   | Requirement/Evidence F1; Exact Set                           | C1 targets; target-level macro                  |
| RQ2   | Attribute Reconstruction; Matched Full-State Exact; Coverage | Matched historical requirements; paired C2–C1   |
| RQ3   | Decision/Balanced Accuracy; ACT and CLARIFY success          | Separate Gold-ACT and Gold-CLARIFY denominators |
| RQ4   | Repository Success Rate; Executable Coverage                 | Eligible targets; C1/C2 common support          |

**Agents and run controls.** We treat the model and its coding-agent harness as a single deployed system because scaffolding, tool use, and stopping behavior can affect the result. The experiment manifest will record the provider model identifier and revision, harness version, evaluation date, context window, temperature and sampling settings, prompt and output-schema hashes, permitted tools and network access, token and wall-clock budgets, retry and stopping policies, and the number of independent runs. The runner will retain the frozen Phase-A response, tool and token logs, final Phase-B repository, and evaluator outputs. *[TODO: freeze the evaluated Codex and Claude Code configurations, exact model versions, prompts, budgets, evaluation dates, and replicate policy before formal runs.]*

**Scoring.** RQ1 aligns predicted and gold requirement atoms through a frozen semantic-relation classifier followed by deterministic one-to-one matching. It reports Requirement Precision, Recall, and F1; Evidence Precision, Recall, and F1; and Exact Requirement Set Accuracy. Requirement and evidence scores remain separate, and RQ1 receives no C2 result.

RQ2 evaluates the pre-task state only for one-to-one matched historical requirements. Typed comparators score attributes, scope, lifecycle, ambiguity, and execution. *Matched Full-State Exact* requires every applicable field of every matched requirement to be correct. Because unmatched gold requirements are not scored a second time, Reconstruction Coverage,  $|M_t|/|R_t^{\text{gold}}|$ , is reported separately. Targets with no matched requirement are N/A, not perfect reconstructions. An oracle-aligned constant-state baseline exposes class imbalance, and the paired C2–C1 contrast estimates the effect of removing irrelevant and stale history.

RQ3 first scores the ACT/CLARIFY decision using Decision Accuracy, Balanced Accuracy, and class recall. Gold-ACT targets additionally report typed Post-State Score, Post-State Exact, and end-to-end success. A missing affected requirement scores zero, and an extra requirement is a closed-world false positive. Gold-CLARIFY targets report blocking-issue Precision/Recall/F1, question validity, and clarification success, with requirement-, dimension-, and field-level diagnostics. Unsupported Autonomy uses Gold-CLARIFY targets as its denominator, whereas Unnecessary Clarification uses Gold-ACT targets. All-ACT and all-CLARIFY decision baselines are reported for both conditions.

RQ4 scores only the final repository. A target–condition pair is eligible only if its frozen RQ3 gold decision is ACT, the pre-task environment is reproducibly runnable, the requested behavior is deterministically observable, the hidden target tests have passed two independent reviews, the validator has passed calibration, and the agent-visible repository passes the leakage audit. For an eligible target,

$$\text{RQ4Pass} = \text{BuildPass} \wedge \text{TargetTestPass} \wedge \text{RegressionPass}.$$

A frozen non-ACT or unparsable Phase-A response on an eligible target yields NO\_CODE\_SUBMISSION/FAIL; evaluator, environment, or harness failures instead invalidate the attempt and trigger a clean rerun. RQ4 reports Success Rate and Executable Coverage. The primary C1/C2 comparison uses only targets eligible under both conditions with the same validator, while all-eligible results and coverage are shown separately.

**Aggregation and uncertainty.** Metrics are first computed at their stated unit and then macro-averaged over targets; benchmark-level cross-project claims use project macro-averages so that projects with many targets do not dominate. Every table reports the number of targets for each

Table 5: Main-result template for RQ1 selection and RQ2 pre-task reconstruction. All entries remain TBD until the formal evaluation is frozen and executed.

| Panel A: RQ1 selection under C1 |         |        |        |         |           |            |            |             |       |
|---------------------------------|---------|--------|--------|---------|-----------|------------|------------|-------------|-------|
| Model/agent                     | Targets | Req. P | Req. R | Req. F1 | Exact Set | Evidence P | Evidence R | Evidence F1 |       |
| [Model/agent]                   | [TBD]   | [TBD]  | [TBD]  | [TBD]   | [TBD]     | [TBD]      | [TBD]      | [TBD]       | [TBD] |

| Panel B: RQ2 reconstruction under C1/C2 |                |              |            |       |           |           |           |               |                 |
|-----------------------------------------|----------------|--------------|------------|-------|-----------|-----------|-----------|---------------|-----------------|
| Model/agent                             | Cond./contrast | N            | Attributes | Scope | Lifecycle | Ambiguity | Execution | Matched Exact | Recon. Coverage |
| [Model/agent]                           | C1             | [TBD]        | [TBD]      | [TBD] | [TBD]     | [TBD]     | [TBD]     | [TBD]         | [TBD]           |
| [Model/agent]                           | C2             | [TBD]        | [TBD]      | [TBD] | [TBD]     | [TBD]     | [TBD]     | [TBD]         | [TBD]           |
| [Model/agent]                           | C2–C1          | [paired TBD] | [TBD]      | [TBD] | [TBD]     | [TBD]     | [TBD]     | [TBD]         | [TBD]           |
| Oracle-aligned constant                 | Baseline       | [TBD]        | [TBD]      | [TBD] | [TBD]     | [TBD]     | [TBD]     | [TBD]         | [TBD]           |

condition and RQ3 branch. C2–C1 effects use paired targets with identical gold, and RQ4 uses the common-support set described above. Class-conditional recall and risk rates with small denominators will include their numerator, denominator, and two-sided exact-binomial 95% confidence interval. Semantic-judge or schema failures are recorded as JUDGE\_ERROR and rerun rather than converted into agent errors. Detailed field scores, judge calibration, validator calibration, and per-project results will be reported in the appendix. [TODO: freeze the replicate aggregation rule, confidence-interval and paired-comparison procedures, multiple-comparison policy, and final handling of agent timeouts and other missing outputs.]

## 5.2 MAIN RESULTS: REQUIREMENT IDENTIFICATION AND RECONSTRUCTION

**RQ1: Can agents select the relevant historical requirements?** [Writing template: Begin with one sentence that answers RQ1 using Requirement F1 and its uncertainty under C1. Identify the strongest model–harness pair and quantify the gap between Requirement F1, Evidence F1, and Exact Requirement Set Accuracy. Then explain the dominant precision or recall failure with one representative case. Report all target counts and avoid generalizing from a single project or history-length stratum.]

**RQ2: Can agents reconstruct the current requirement state?** [Writing template: Begin with one sentence that answers RQ2 using Attribute Reconstruction and Matched Full-State Exact. Report the paired C2–C1 difference with uncertainty, followed by Reconstruction Coverage so that matched-only state quality is not mistaken for end-to-end selection performance. Compare against the oracle-aligned constant-state baseline, identify the weakest state dimension, and discuss a counterexample. Do not apply current-task updates to the pre-task state.]

## 5.3 MAIN RESULTS: UPDATE DECISIONS AND CODE DELIVERY

**RQ3: Can agents produce the correct requirement update or clarification?** [Writing template: Answer RQ3 first with Decision Accuracy and Balanced Accuracy under C1/C2, together with Gold ACT/CLARIFY counts and the all-ACT/all-CLARIFY baselines. Report paired C2–C1 effects before separating the branches. For Gold-ACT targets, report Post-State Exact and end-to-end success; for Gold-CLARIFY targets, report Blocking-Issue F1, Question Validity, and Clarification Success. State the numerator, denominator, and exact 95% interval for Unsupported Autonomy and Unnecessary Clarification, and do not combine their denominators.]

**RQ4: Can agents translate requirements into correct code?** [Writing template: Answer RQ4 with repository Success Rate on the C1/C2 common-support set, including paired uncertainty. Then report each condition’s all-eligible Success Rate and Executable Coverage with Gold-ACT, eligible, and common-support counts. Attribute a history-condition effect only on common support. Reserve Build, Target Test, and Regression pass rates for failure analysis, and do not report formal PASS/FAIL values until the RQ3 gold, eligibility gates, validators, calibration, and leakage audits are frozen.]

Table 6: Main-result template for RQ3 update-or-clarify reasoning and RQ4 repository delivery. All entries remain TBD until the formal evaluation is frozen and executed.

| Panel A: RQ3 decision quality |       |           |           |           |          |              |                 |                |
|-------------------------------|-------|-----------|-----------|-----------|----------|--------------|-----------------|----------------|
| Model/agent                   | Cond. | Gold A/C  | Dec. Acc. | Bal. Acc. | ACT Rec. | CLARIFY Rec. | Unsup. Autonomy | Unnec. Clarify |
| [Model/agent]                 | C1    | [TBD/TBD] | [TBD]     | [TBD]     | [TBD]    | [TBD]        | [TBD]           | [TBD]          |
| [Model/agent]                 | C2    | [TBD/TBD] | [TBD]     | [TBD]     | [TBD]    | [TBD]        | [TBD]           | [TBD]          |
| All-ACT                       | C1/C2 | [TBD/TBD] | [TBD]     | [TBD]     | [TBD]    | [TBD]        | [TBD]           | [TBD]          |
| All-CLARIFY                   | C1/C2 | [TBD/TBD] | [TBD]     | [TBD]     | [TBD]    | [TBD]        | [TBD]           | [TBD]          |

| Panel B: RQ3 Gold-ACT branch |       |              |                  |                  |                    |
|------------------------------|-------|--------------|------------------|------------------|--------------------|
| Model/agent                  | Cond. | Gold-ACT $N$ | Post-State Score | Post-State Exact | End-to-End Success |
| [Model/agent]                | C1    | [TBD]        | [TBD]            | [TBD]            | [TBD]              |
| [Model/agent]                | C2    | [TBD]        | [TBD]            | [TBD]            | [TBD]              |

| Panel C: RQ3 Gold-CLARIFY branch |       |                  |              |              |               |           |           |            |                   |                       |
|----------------------------------|-------|------------------|--------------|--------------|---------------|-----------|-----------|------------|-------------------|-----------------------|
| Model/agent                      | Cond. | Gold-CLARIFY $N$ | Req. Correct | Dim. Correct | Field Correct | Blocker P | Blocker R | Blocker F1 | Question Validity | Clarification Success |
| [Model/agent]                    | C1    | [TBD]            | [TBD]        | [TBD]        | [TBD]         | [TBD]     | [TBD]     | [TBD]      | [TBD]             | [TBD]                 |
| [Model/agent]                    | C2    | [TBD]            | [TBD]        | [TBD]        | [TBD]         | [TBD]     | [TBD]     | [TBD]      | [TBD]             | [TBD]                 |

| Panel D: RQ4 repository delivery |       |              |              |            |               |                 |                |
|----------------------------------|-------|--------------|--------------|------------|---------------|-----------------|----------------|
| Model/agent                      | Cond. | Gold-ACT $N$ | Eligible $N$ | Common $N$ | PASS (common) | PASS (eligible) | Exec. Coverage |
| [Model/agent]                    | C1    | [TBD]        | [TBD]        | [TBD]      | [TBD]         | [TBD]           | [TBD]          |
| [Model/agent]                    | C2    | [TBD]        | [TBD]        | [TBD]      | [TBD]         | [TBD]           | [TBD]          |

## 6 ANALYSIS AND LIMITATIONS

This section will analyze concrete failure mechanisms rather than only model rankings. Each implication will be tied to an observed result or case.

### 6.1 FAILURE MODES AND ERROR PROPAGATION

**Requirement-evolution failure modes.** The analysis will cover `INTRODUCE`, `MODIFY`, `DEFER`, `RESUME`, `REMOVE`, `AMBIGUOUS`, `IMPLEMENTATION_CLAIM`, `RUNTIME_FAILURE`, and `RUNTIME_VERIFICATION`. `CLARIFY` will be analyzed as an RQ3 decision rather than a Requirement Event. Initial categories include stale-state errors that retain superseded values, resurrection errors that reuse removed requirements, missing-scope errors, ambiguity hallucinations, and execution-state confusion. *[TODO: revise this taxonomy using observed errors and add anonymized, provenance-safe examples.]*

**Effects of history characteristics.** Analyze history length, evidence distance, relevant-evidence density, number of updates, parallel requirements, target position, and number of affected requirements. The final analysis will specify bins, controls, model, and multiple-comparison treatment before interpreting correlations.

**Does unfiltered history hurt?** Compare C1 Full History with C2 Oracle Relevant History on paired targets with the same gold. A C2–C1 difference will be attributed to removal of irrelevant and stale context only after confirming identical prompts, models, budgets, non-repository tools, and sufficient oracle evidence. Candidate mechanisms include noise, stale-state interference, truncation, context competition, and dimension-specific state errors. No-History remains outside the formal comparison.

**Coding ability versus requirement-state ability.** Compare model rankings on a standard coding benchmark, ReqMemBench pre/post-state reasoning, and ReqMemBench RQ4 delivery. Any ranking inversion will be interpreted only after aligning model versions, evaluation dates, harnesses, and uncertainty. If no inversion occurs, the final text will report the observed relationship directly.

**Error propagation across the four stages.** The analysis distinguishes

MemoryFailure  $\neq$  StateReasoningFailure  $\neq$  DecisionFailure  $\neq$  ImplementationFailure.

Table 7: Planned cross-stage failure analysis.

| Stage | Diagnostic question                                              | Typical failure                                                                     |
|-------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| RQ1   | Were the affected historical requirements and evidence selected? | Missed gold atom/evidence group or imported unrelated requirement                   |
| RQ2   | Were the matched pre-task states reconstructed?                  | Stale attributes, widened scope, ignored life-cycle, or confused execution evidence |
| RQ3   | Was the complete update or every material blocker identified?    | Incorrect update, unsupported action, or unnecessary/mislocalized question          |
| RQ4   | Did the eligible ACT run pass the frozen validator?              | Correct reasoning followed by Build, Target Test, or Regression failure             |

**Figure 4 placeholder: cross-stage error propagation**

Conditional RQ4 PASS rates given correct versus incorrect RQ2/RQ3 outputs, optionally paired with the strongest verified history-characteristic effect.

Figure 4: Planned diagnostic of how requirement-state and decision errors propagate to repository delivery.

## 6.2 IMPLICATIONS AND LIMITATIONS

**Implications for coding-agent design.** The final discussion will retain only implications supported by measured evidence. Candidate implications are that memory should represent current state rather than only retrieve messages; invalidation can matter as much as remembering; requirement memory should preserve temporal provenance; material unresolved ambiguity should trigger clarification; and requirement reasoning should be frozen before repository execution so code cannot reveal the state-reconstruction answer.

**Limitations.** The final paper will cover:

1. **Data domain.** Freelance projects do not represent every enterprise process, organization, safety requirement, or codebase.
2. **Data-generation provenance.** If histories or code states are rewritten, reconstructed, simulated, or mixed, conclusions apply to that construction rather than to unqualified naturally occurring development.
3. **Historical takeover setting.** ReqMemBench supplies an observed history and does not directly evaluate an online memory-writing policy from  $t_0$ .
4. **Annotation subjectivity.** Requirement atomicity, typed state fields, material ambiguity, post-task branches, and clarification targets permit interpretation differences and require stage-specific reliability evidence.
5. **Code availability and executability.** Dependencies, environments, external services, or original snapshots may not be recoverable for every project.
6. **Requirement gold versus implementation gold.** Multiple implementations may satisfy one requirement:  

$$\text{RequirementGold} \neq \text{UniqueCodePatch}.$$
7. **Evaluation-judge dependence.** Semantic scorers, target selectors, and validators may introduce model or schema bias; calibration and deterministic checks bound but do not eliminate this risk.
8. **Sampling and dependence.** Multiple targets from one project are correlated, and empty or imbalanced history strata limit generalization.
9. **Coverage.** The benchmark may not cover multi-team enterprise development, combined issue-tracker/chat/code-review histories, multi-year memory, or continuously operating autonomous agents.

## 7 CONCLUSION

The conclusion will briefly state what existing evaluation misses, what ReqMemBench introduces, and the one or two most important measured findings. ReqMemBench evaluates whether an agent can identify directly affected historical requirements, reconstruct their pre-task states, update or clarify affected post-task states, and complete eligible repository deliveries at intermediate takeover points. *[TODO: insert the final findings and end with the supported form of the central message: persistent coding agents must know both how to code and what the project requires now.]*

### AI USE STATEMENT

*[TODO: complete this statement according to the current ICLR 2027 author policy; disclose generative-AI use in research ideation, data processing, annotation, code, experiments, writing, or editing; and explain human verification and responsibility.]*

### ETHICS STATEMENT

*[TODO: describe data authorization, Terms-of-Service or contractual constraints, privacy and de-identification, credentials and sensitive content, reconstructed or simulated artifacts, release scope, re-identification risk, potential harms, and safeguards. Distinguish source-data access from the public release.]*

### REPRODUCIBILITY STATEMENT

*[TODO: point to the sections, appendices, code, configurations, prompts, schemas, dataset cards, and supplementary artifacts covering selection, provenance, PII processing, annotation, replay, target construction, instance materialization, models, metrics, judge calibration, validator calibration, and statistics.]*

## REFERENCES

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>. MBPP.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>. HumanEval.
- Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, 2023. doi: 10.52202/075280-2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/920f2dced7d32ab2ba2f1970bc306af6-Abstract-Datasets\\_and\\_Benchmarks.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/920f2dced7d32ab2ba2f1970bc306af6-Abstract-Datasets_and_Benchmarks.html).

- O. C. Z. Gotel and Anthony Finkelstein. An analysis of the requirements traceability problem. In *Proceedings of the First IEEE International Conference on Requirements Engineering*, pp. 94–101. IEEE Computer Society, 1994. doi: 10.1109/ICRE.1994.292398.
- Hojae Han, Seung-won Hwang, Rajhans Samdani, and Yuxiong He. ConvCode-World: Benchmarking conversational code generation in reproducible feedback environments. In *International Conference on Learning Representations (ICLR)*, 2025. URL [https://proceedings.iclr.cc/paper\\_files/paper/2025/hash/6091f2bb355e960600f62566ac0e2862-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2025/hash/6091f2bb355e960600f62566ac0e2862-Abstract-Conference.html).
- Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025. URL <https://arxiv.org/abs/2507.05257>. MemoryAgentBench.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html).
- Thomas Joshi, Shayan Chowdhury, and Fatih Uysal. SWE-Bench-CL: Continual learning for coding agents. *arXiv preprint arXiv:2507.00014*, 2025.
- Tue Le, Minh Vu Thai Pham, Dung Nguyen Manh, Huy Nhat Phan, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025. v6 revised May 2026 – CHECK: set year to 2026 if you cite the revision or a subsequent venue version.
- Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. In *International Conference on Learning Representations (ICLR)*, 2024. URL [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/d191ba4c8923ed8fd8935b7c98658b5f-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/d191ba4c8923ed8fd8935b7c98658b5f-Abstract-Conference.html).
- Yang Liu, Li Zhang, Fang Liu, Ping Lin, and Xinyi Li. A scalable benchmark for repository-oriented long-horizon conversational context management. *arXiv preprint arXiv:2603.06358*, 2026. LoCoEval.
- Pengrui Lu, Shiqi Zhang, Yunzhong Hou, Lyumanshan Ye, Chaoyi Huang, Zixi Chen, Ji Zeng, Hantao Jiang, Pengfei Liu, Yiwei Wang, and Ming-Hsuan Yang. ProjDevBench: Benchmarking AI coding agents on end-to-end project development. *arXiv preprint arXiv:2602.01655*, 2026.
- Meher Bhaskar Madiraju and Meher Sai Preetam Madiraju. RigorBench: Benchmarking engineering process discipline in autonomous AI coding agents. *arXiv preprint arXiv:2606.22678*, 2026. Unrefereed preprint, 30 tasks – see notes on citation weight.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- Chunyu Miao, Henry Peng Zou, Yangning Li, Yankai Chen, Yibo Wang, et al. RECODE-H: A benchmark for research code development with interactive human feedback. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.06186.
- Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can frontier LLMs earn \$1 million from real-world freelance software engineering? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 44412–44450. PMLR, 2025. URL <https://proceedings.mlr.press/v267/miserendino25a.html>.
- Bashar Nuseibeh and Steve M. Easterbrook. Requirements engineering: A roadmap. In *Proceedings of the 22nd International Conference on Software Engineering, Future of Software Engineering Track*, pp. 35–46. ACM, 2000. doi: 10.1145/336512.336523.

- Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*, 2026.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL [https://proceedings.iclr.cc/paper\\_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html).
- Zexun Zhan, Shuzheng Gao, Ruida Hu, and Cuiyun Gao. SR-Eval: Evaluating LLMs on code generation under stepwise requirement refinement. *arXiv preprint arXiv:2509.18808*, 2025.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A benchmark for context learning in coding. *arXiv preprint arXiv:2602.08316*, 2026. Official name is "SWE Context Bench"; not the same paper as ContextBench (arXiv:2602.05892).
- Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.15877.

## A APPENDIX STRUCTURE

The appendix will contain:

1. the complete requirement-state schema and field definitions;
2. the event taxonomy and state-transition rules;
3. project selection, provenance categories, authorization, privacy, and de-identification procedures;
4. annotation guidelines, review interfaces, and agreement analysis;
5. target-time selection and leakage controls;
6. expanded RQ1–RQ4 definitions and examples, complete prompts, public response schemas, and hidden scoring contracts;
7. model, agent, tool, budget, and retry configurations;
8. full metric equations, field-level comparators, branch-specific denominators, aggregation, uncertainty, and statistical tests;
9. judge and validator calibration, baselines, and human-ceiling estimates;
10. full benchmark distributions and eligibility counts;
11. full per-RQ result tables and breakdowns by project, event, condition, model, and history characteristic;
12. failure cases and qualitative examples;
13. one end-to-end example from source evidence through events, the state graph, target-local pre/post states, the frozen Phase-A output and score, and validated repository delivery; and
14. a data card and release statement explaining which artifacts can and cannot be shared.